

# Architecture des Données - Dual Book Advisor

## Deux Bases Complémentaires

### Base Django (db.sqlite3)

- **Rôle** : Stockage persistant et source de vérité
- **Contenu** : Métadonnées complètes des livres
- **Localisation** : `db.sqlite3` (racine du projet)
- **Accès** : Via Django ORM (`LiteratureBook.objects.all()`)

python

*# Exemple : Voir tous les mangas en base Django*

```
from apps.books.models import LiteratureBook
mangas = LiteratureBook.objects.filter(categories__icontains='Manga')
print(f"Mangas en base Django: {mangas.count()}")
```

### Base ChromaDB (chroma\_db/)

- **Rôle** : Recherche sémantique/vectorielle
- **Contenu** : Embeddings + métadonnées de recherche
- **Localisation** : `rag/literature_rag/chroma_db/`
- **Accès** : Via RAG Manager pour recherche IA

python

*# Exemple : Recherche sémantique*

```
from rag.literature_rag.literature_rag_manager import LiteratureRAGManager
rag = LiteratureRAGManager()
results = rag.search_books("manga adventure", n_results=3)
```

## Processus Complet

### Étape 1 : Import

CSV manga → Django DB (LiteratureBook)

- Les mangas sont stockés dans `db.sqlite3`
- Chaque manga = une ligne dans la table `books_literaturebook`

### Étape 2 : Indexation

Django DB → ChromaDB (embeddings)

- Le RAG Manager lit tous les `LiteratureBook`
- Génère des embeddings des descriptions
- Stocke dans ChromaDB pour recherche rapide

## Étape 3 : Recherche IA

Requête utilisateur → ChromaDB → IDs des livres → Django DB → Données complètes

### Vérifications Possibles

#### Vérifier Django DB

bash

```
python manage.py shell
>>> from apps.books.models import LiteratureBook
>>> print(f"Total livres: {LiteratureBook.objects.count()}")
>>> print(f"Mangas: {LiteratureBook.objects.filter(categories__icontains='Manga').count()}")
```

#### Vérifier ChromaDB

bash

```
python -c "
from rags.literature_rag.literature_rag_manager import LiteratureRAGManager
rag = LiteratureRAGManager()
stats = rag.get_stats()
print(f'ChromaDB: {stats}')
"
```

### Pourquoi cette Architecture ?

1. **Django DB** : Stockage fiable, relations, admin interface
2. **ChromaDB** : Recherche IA ultra-rapide, similarité sémantique
3. **Sync** : ChromaDB se synchronise avec Django DB

### Résolution de Problèmes

**Si mangas visibles dans Django mais pas dans recherche :**

```
bash
```

```
# Réindexer ChromaDB
```

```
python scripts/setup_rag.py --literature-only --reset
```

## **Si mangas pas dans Django :**

```
bash
```

```
# Réimporter
```

```
python scripts/import_manga_no_telemetry.py
```